

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

arXiv:2305.10601

Feliciann Elliot

MAI 5301 - Foundations Of Large
Language Models

Abstract

Large Language Models (LLMs) typically generate responses through linear, token-by-token decoding, which limits their ability to plan, revise, or recover from early reasoning errors. The Tree of Thoughts (ToT) framework proposes an alternative inference-time approach that enables structured exploration over multiple intermediate reasoning states. Instead of committing to a single chain of thought, the model generates, evaluates, and prunes multiple candidate “thoughts” using search algorithms such as breadth-first and depth-first search. This allows the model to backtrack from unproductive paths and compare alternative partial solutions before producing a final answer.

Experimental results demonstrate significant performance improvements on tasks requiring long-horizon reasoning, planning, and constraint satisfaction. The paper argues that structured reasoning and search can yield greater gains than scaling model size alone, while also introducing new computational tradeoffs.

Introduction

Standard LLM inference relies on autoregressive decoding, forcing models to commit to early reasoning decisions that cannot be revised. While Chain-of-Thought prompting exposes intermediate steps, it still follows a single, irreversible reasoning path. Many complex tasks require exploration, comparison of alternatives, and backtracking—capabilities absent in conventional LLM inference. Tree of Thoughts addresses this limitation by introducing explicit search over reasoning states, enabling more deliberate and globally planned problem solving.

What Is Tree of Thoughts?

Tree of Thoughts (ToT) is an inference-time framework that replaces linear decoding with structured search over intermediate reasoning states.

- Each node represents a coherent “thought” rather than a single token.
- The model explicitly explores, evaluates, and prunes reasoning paths before committing to an answer.

Why Tree of Thoughts Was Created

- Standard autoregressive decoding forces irreversible early decisions.
- Chain-of-Thought still commits to a single reasoning trajectory.
- Many tasks require exploration, backtracking, and long-horizon planning.

The Core Problem

LLM inference lacks global planning capabilities, operating purely through local token-by-token decisions.

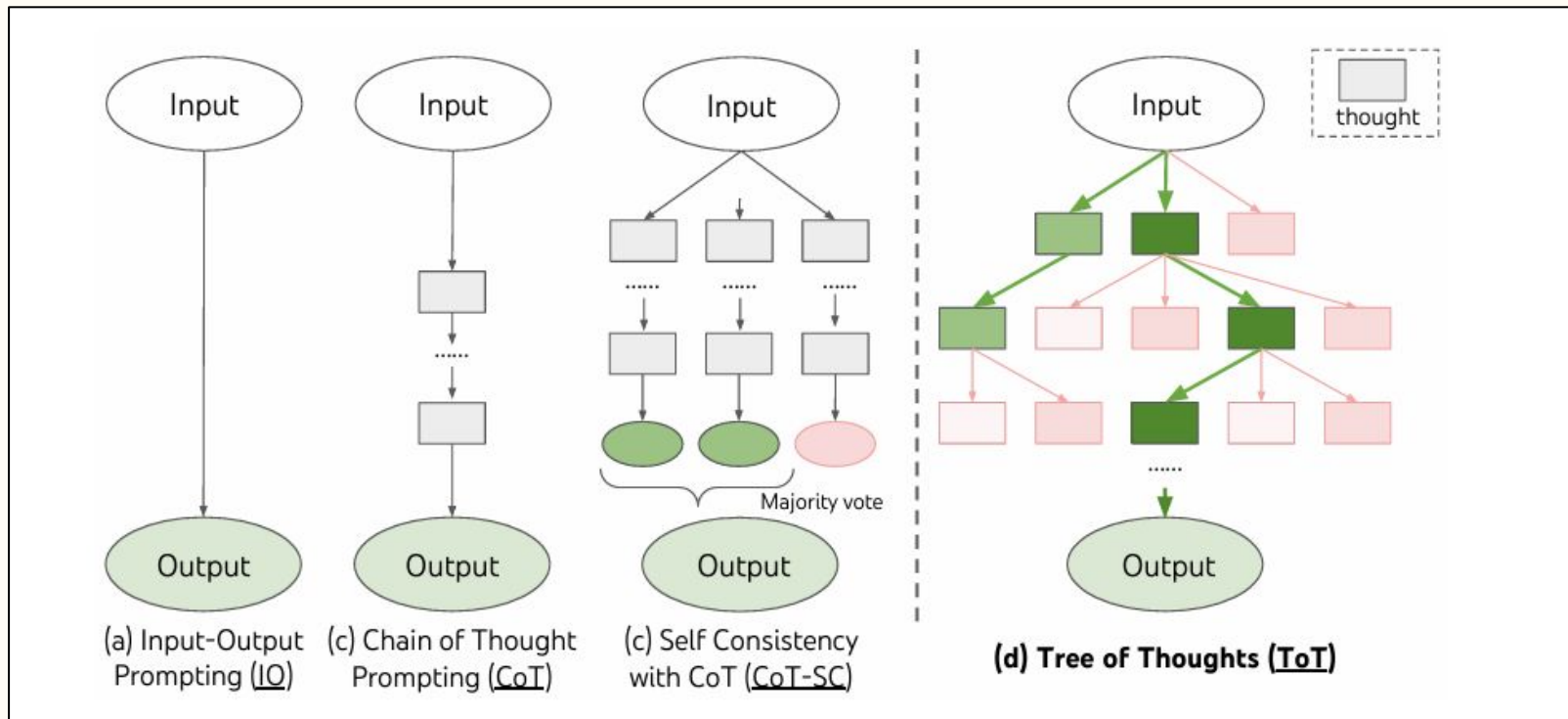
There is no mechanism for revising early reasoning mistakes once they are generated, and no structured way to compare alternative partial solutions before committing to a complete answer.

Related Works

Method	Core Idea	Key Limitation	References
IO Prompting	Direct mapping from input to output without intermediate reasoning	No visibility into reasoning; brittle on multi-step tasks	Brown et al., 2020 (GPT-3); Chowdhery et al., 2022 (PaLM)
Chain-of-Thought	Explicit linear reasoning steps before final answer	Single reasoning path; errors compound with no recovery	Wei et al., 2022
Self-Consistency	Sample multiple CoT paths and select the majority answer	No correction or exploration within a reasoning path	Wang et al., 2022
Reflexion / Self-Re	Iterative feedback and refinement of previous outputs	Still sequential; lacks structured search or branching	Shinn et al., 2023; Madaan et al., 2023

IO vs Chain of Thought vs Self-Consistency

CoT vs Tree of Thought



What Makes Tree of Thoughts Different

ToT introduces an explicit tree structure over reasoning states rather than linear generation.

It decouples generation, evaluation, and search into separate processes, allowing the model to generate candidate thoughts, evaluate their promise, and navigate the search space independently.

Tree of Thoughts: Core Components

Tree of Thoughts is build on four core components:

- . Thought Representation
- . Thought Generation
- . State Evaluation
- . Search and Backtracking Strategy

Together, these components enable deliberate exploration rather than linear reasoning.

Component: Thought Representation

- A “thought” is a semantically meaningful unit (equation, plan, word).
- They are larger than tokens, smaller than full answers.
- They enable evaluable intermediate states.

Component: Thought Representation

	Game of 24	Creative Writing	5x5 Crosswords
Input	4 numbers (4 9 10 13)	4 random sentences	10 clues (h1. presented;..)
Output	An equation to reach 24 (13-9)*(10-4)=24	A passage of 4 paragraphs ending in the 4 sentences	5x5 letters: SHOWN; WIRRA; AVAIL; ...
Thoughts	3 intermediate equations (13-9=4 (left 4,4,10); 10-4=6 (left 4,6); 4*6=24)	A short writing plan (1. Introduce a book that connects...)	Words to fill in for clues: (h1. shown; v5. naled; ...)
#ToT steps	3	1	5-10 (variable)

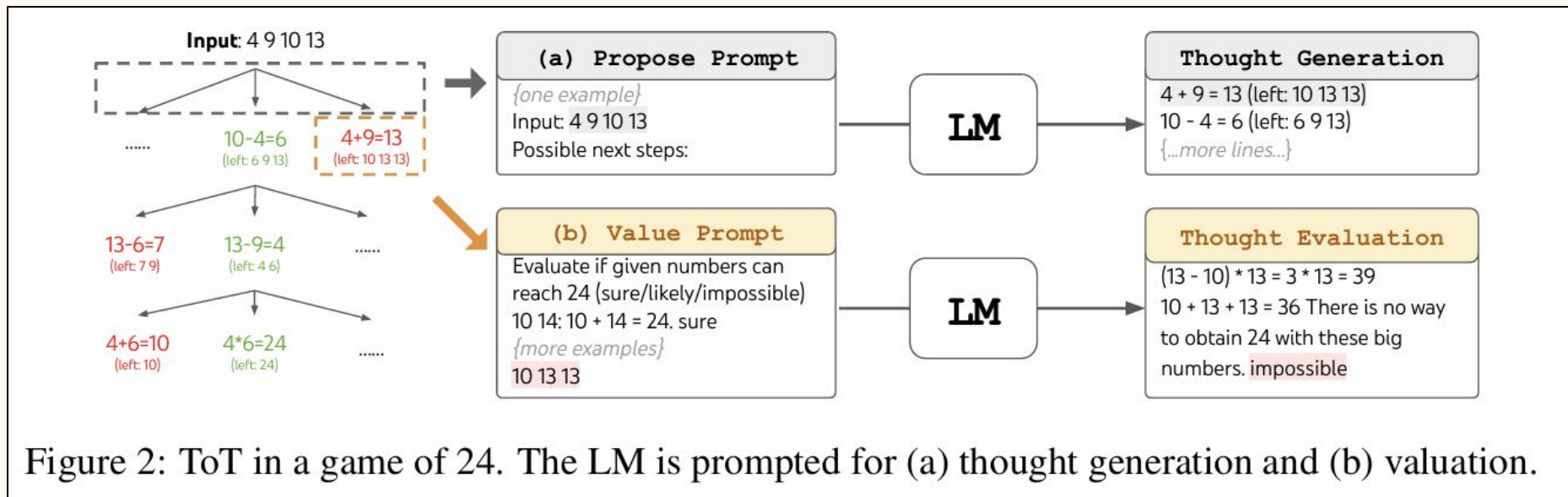
Table 1: Task overview. Input, output, thought examples are in blue.

Component: Thought Generation

Thought generation is the process where the language model produces multiple candidate “thoughts” at each reasoning step, rather than a single next token or single next step.

These thoughts represent possible next reasoning states in the search tree.

Component: Thought Generation



Component: Thought Generation

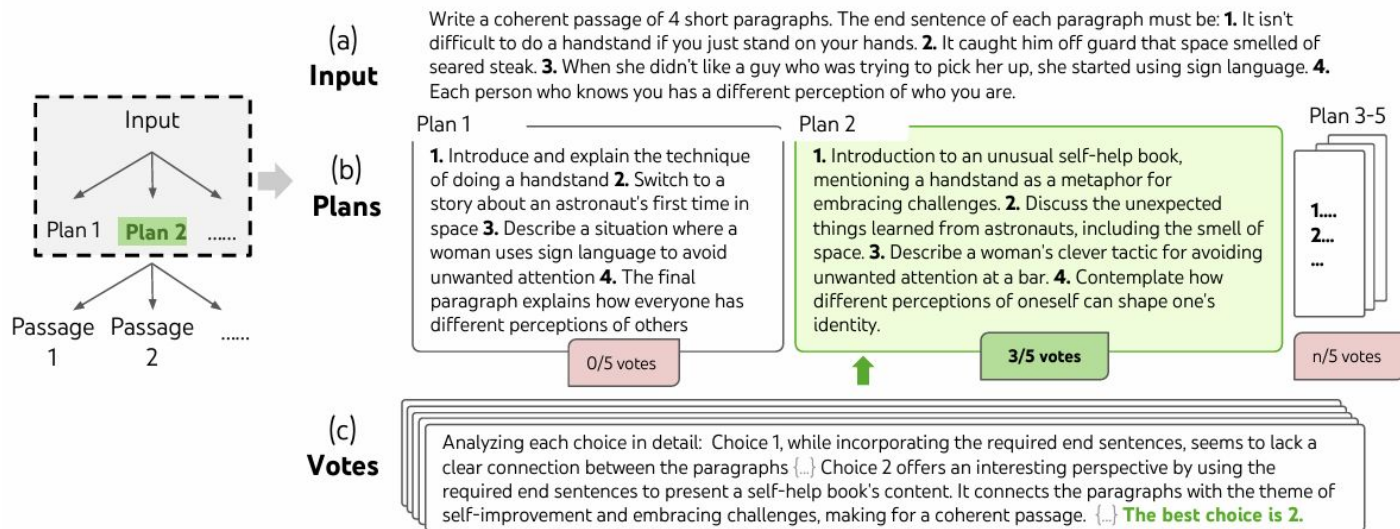


Figure 4: A step of deliberate search in a randomly picked Creative Writing task. Given the input, the LM samples 5 different plans, then votes 5 times to decide which plan is best. The majority choice is used to consequently write the output passage with the same sample-vote procedure.

Component: State Evaluation (Heuristic Scoring)

The language model evaluates its own intermediate states.

This self-evaluation acts as a learned heuristic to guide search, distinguishing promising reasoning paths from unproductive ones without requiring external supervision or ground-truth feedback.

Component: Search and Backtracking Strategy

- BFS (Breadth-First Search): Applied to shallow, branching tasks (Game of 24) requiring broad exploration at each step.
- DFS with pruning (Depth-First Search): Used for deep search tasks (Crosswords) where solutions require long reasoning chains and early pruning of unpromising paths.

Component: Search and Backtracking Strategy

- Backtracking mechanism: Enables recovery from dead ends by reverting to earlier states and exploring alternative branches.

Component: Search and Backtracking Strategy

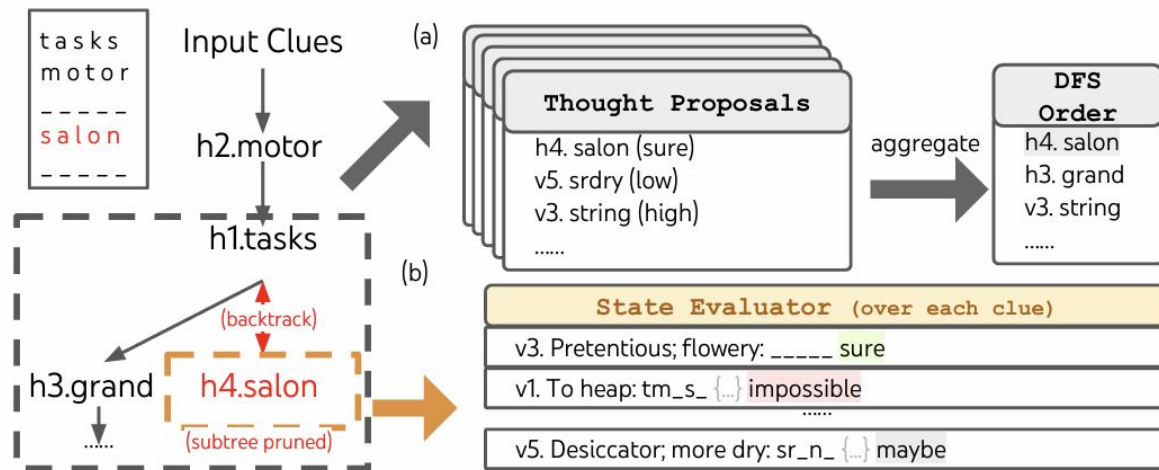


Figure 6: In Mini Crosswords, (a) how thoughts are proposed and aggregated in a priority queue for depth-first search (DFS), and (b) how a state is evaluated based on the possibility of filling in each remaining word clue, and pruned if any remaining clue is deemed not possible to fill by the LM. Then DFS backtracks to the parent state and explore the next promising thought for clue.

Evaluation Tasks

- Game of 24 (symbolic reasoning)
- Creative Writing (planning + coherence)
- Mini Crosswords (long-horizon constraint satisfaction)

Performance Results

- Game of 24: 74% success vs <10% for CoT.
- Creative writing shows higher coherence (human + GPT-4 eval).
- Crosswords demonstrate necessity of backtracking.

Performance Results

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC ($k=100$)	9.0%
ToT (ours) ($b=1$)	45%
ToT (ours) ($b=5$)	74%
IO + Refine ($k=10$)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

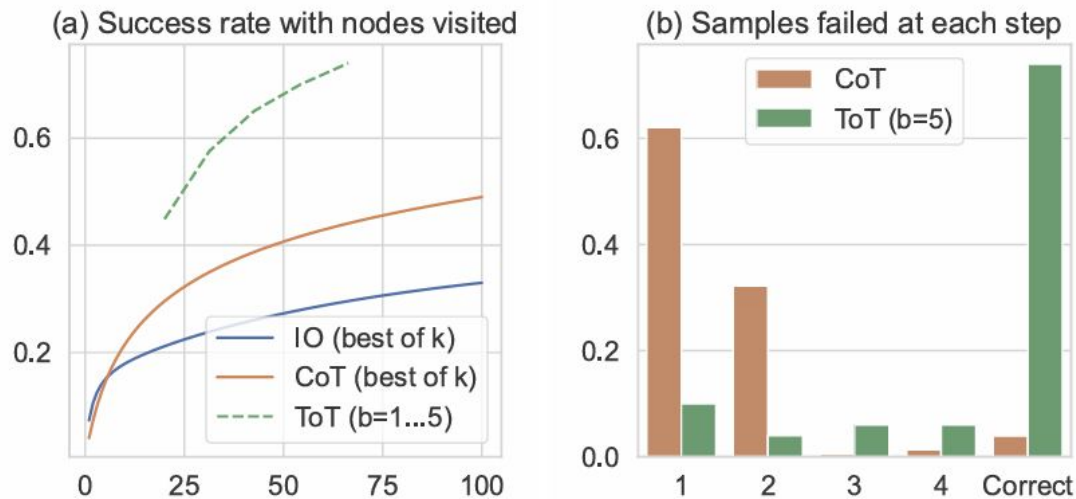


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

Inference Cost vs Performance

- Tree of Thought uses more tokens than Chain of Thought.
- Comparable to “best-of-k” sampling but higher success.
- Cost-performance tradeoff is configurable.

Inference Cost vs Performance

Game of 24	Generate/Prompt tokens	Cost per case	Success
IO (best of 100)	1.8k / 1.0k	\$0.13	33%
CoT (best of 100)	6.7k / 2.2k	\$0.47	49%
ToT	5.5k / 1.4k	\$0.74	74%

Table 7: Cost analysis on Game of 24.

On Creative Writing (Table 8 below), we found ToT takes around 5x completion tokens and money cost, which is intuitive as $b = 5$ and most tokens are generated passages.

Creative Writing	Generate/Prompt tokens	Cost per case
IO	0.9k / 0.4k	\$0.06
CoT	0.9k / 0.4k	\$0.07
ToT	4k / 2.9k	\$0.32

Table 8: Cost analysis on Game of 24.

Tree of Thought Limitations

- Computationally expensive at inference, requiring multiple model calls for generation, evaluation, and search.
- Heuristic evaluation can be imperfect, occasionally misranking promising reasoning paths.
- Performance bounded by the base model underlying reasoning quality and cannot overcome fundamental limitations.

Future Research

- Training models explicitly for thought evaluation rather than relying on zero-shot heuristics.
- Hybrid search methods incorporating A* or Monte Carlo Tree Search (MCTS) for more efficient exploration.
- Combining smaller models with structured reasoning to reduce computational cost while maintaining performance.

Conclusion

Structured reasoning architectures that enable exploration, evaluation, and backtracking may yield larger performance gains on complex tasks than simply scaling model size.

The ability to systematically search solution spaces can be more valuable than raw generative capability alone.

References

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023, May 17). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv.org.
<https://arxiv.org/abs/2305.10601>



Questions?

Let's Verify Step by Step

arXiv:2305.20050

Feliciann Elliot

MAI 5301 - Foundations Of Large
Language Models

Abstract

Although Large Language Models can generate fluent multi-step reasoning, correctness is not guaranteed, and errors often go undetected within long chains of thought. The paper Let's Verify Step by Step proposes a verification-based framework that separates reasoning generation from correctness judgment. Instead of trusting end-to-end solutions, the model decomposes generated answers into individual reasoning steps and verifies each step independently. The central insight is that verification is computationally easier than generation, and language models perform more reliably as critics than as solvers. By filtering out incorrect solutions rather than attempting to repair them, this approach yields substantial accuracy improvements on mathematical and logical reasoning benchmarks. The work highlights verification as a complementary mechanism to generative reasoning, emphasizing reliability and error detection over exploration.

Introduction

LLMs can generate fluent multi-step reasoning, however, correctness is not guaranteed. Chain-of-Thought increases transparency but often amplifies confidence in incorrect reasoning. Errors introduced early in a solution frequently propagate without detection, and final-answer accuracy provides no insight into internal reasoning failures. Step-by-Step Verification addresses this issue by explicitly checking each reasoning step, shifting the focus from persuasive reasoning to reliable validation.

What Is Step-by-Step Verification?

- A framework where models generate solutions and then verify each reasoning step.
- Separates reasoning generation from correctness judgment.
- It is inspired by human proof checking.

Let's Verify Step by Step

Tree of Thought asks:

- Which reasoning path should I explore?

Verification asks:

- Is this reasoning actually valid?

Why Verification Was Needed

- Large Language Models can produce fluent but incorrect reasoning.
- Chain-of-Thought increases persuasiveness, not correctness.
- Errors propagate silently.

The Core Problem

- Language models lack an explicit mechanism to detect invalid intermediate reasoning steps during generation.
- End-to-end accuracy metrics hide internal reasoning failures, providing no signal about where errors occur within multi-step solutions.
- Models overtrust their own outputs, propagating mistakes forward without self-correction or uncertainty awareness.

Approach Overview

The system generates candidate solutions to a problem, then decomposes each solution into individual reasoning steps.

Each step is verified independently to identify errors at a granular level, enabling precise localization of reasoning failures rather than binary accept/reject judgments on complete solutions.

Key Insight of Verification

- Verification is computationally and cognitively easier than generation for language models.
- Models demonstrate stronger capability as critics evaluating existing reasoning than as solvers producing novel solutions.
- Accuracy improvements come from filtering out incorrect solutions rather than attempting to fix flawed reasoning, leveraging the asymmetry between verification and generation difficulty.

Evaluation Evidence

- Large gains on math and reasoning benchmarks.
- Verification reduces hallucinated steps.
- Works across model sizes.

Limitations

- Verification accuracy depends on decomposition quality.
- The method filters existing solutions but does not explore alternative reasoning paths or generate corrections for identified errors.
- Cost increases with step count.

Conclusion

- Tree of Thought explores the reasoning space.
- Verification audits the reasoning validity.
- Verification is complementary to Tree of Thought, not competing.

References

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023, May 31). Let's verify step by step. arXiv.org.
<https://arxiv.org/abs/2305.20050>



Questions?